图像闭运算

图像闭运算
- 例程效果
- 原理讲解
  - 什么是图像闭运算
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置闭操作参数
    - 图像处理与闭运算
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/14.rgb888_close.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现 RGB888 格式的图像闭运算处理。



原理讲解

什么是图像闭运算

图像闭运算（Closing）是一种形态学图像处理操作，用于去除图像中的小黑洞（噪声）或连接断开的区域，同时保持图像的整体形状不变。它是膨胀（Dilation）和腐蚀（Erosion）操作的组合，具体顺序为先膨胀后腐蚀。

- **膨胀（Dilation）**：该操作通过将图像中的白色像素（前景）向周围扩展，填充小孔洞、桥接断开的部分，并增强对象的边界。数学上，它使用一个卷积核（通常是矩形或十字形）扫描图像，如果卷积核内有至少一个白色像素，则将中心像素设置为白色。
- **腐蚀（Erosion）**：该操作通过缩小白色区域，移除图像边缘的噪声或细小结构。它要求卷积核内所有像素均为白色时，才将中心像素设置为白色，从而"侵蚀"掉小的孤立白色区域。
- **闭运算的效果**：闭运算 = 膨胀（原图像）+ 腐蚀（膨胀结果）。这使得图像能够去除细小的黑色噪声，同时连接相邻的白色区域。闭运算常用于处理二值图像或在预处理阶段增强对象的连通性，例如在目标检测中去除噪声或在医学图像中填充空洞。

在实际应用中，闭运算的参数（如卷积核大小和迭代次数）会影响处理的强度，而二值化阈值则决定图像的二值化处理方式（如果需要）。

代码概述

导入模块

```
import time, os, gc
from machine import Pin
from media.sensor import *    # 摄像头接口 / Camera interface
from media.display import *   # 显示接口 / Display interface
from media.media import *     # 媒体资源管理器 / Media manager
import cv_lite                # cv_lite 扩展模块（含闭操作）/ AI CV extension (close
function)
import ulab.numpy as np       # NumPy-like ndarray for MicroPython
```

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(width=1280, height=960)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.RGB888)  # 设置像素格式为 RGB888 / Set pixel format to
RGB888
```

- 初始化摄像头，设置初始分辨率为 1280x960（帧率未指定，使用默认值）。
- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合处理彩色图像的形态学操作）。

初始化显示模块

```
Display.init(Display.ST7701, width=image_shape[1], height=image_shape[0],
to_ide=True)
```

初始化显示模块，使用 ST7701 驱动芯片，分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置闭操作参数

```
kernel_size = 3          # 卷积核尺寸（建议为奇数）/ Kernel size (recommended odd)
iterations = 1           # 操作迭代次数 / Number of morphological passes
threshold_value = 100    # 二值化阈值（0=使用 Otsu 自动阈值）/ Threshold for
binarization (0 = Otsu)
clock = time.clock()     # 启动帧率计时器 / Start FPS timer
```

- `kernel_size`：卷积核尺寸，用于形态学操作（膨胀和腐蚀），建议为奇数，值越大处理范围越大。
- `iterations`：闭运算的迭代次数，表示重复执行膨胀和腐蚀的次数，值越大效果越显著。
- `threshold_value`：二值化阈值，用于在闭运算前或后处理图像，值为 0 时使用 Otsu 自动阈值算法。
- `clock`：用于计算帧率。

图像处理与闭运算

```
while True:
    clock.tick()  # 开始计时 / Start frame timing

    # 获取图像帧 / Capture a frame
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()  # 获取 RGB888 ndarray 引用 / Get RGB888 ndarray
reference (HWC)

    # 应用闭操作（先膨胀后腐蚀）/ Apply closing (dilate then erode)
    result_np = cv_lite.rgb888_close(
        image_shape,
        img_np,
        kernel_size,
        iterations,
        threshold_value
    )

    # 构造图像对象（灰度图）用于显示 / Wrap processed grayscale image for display
    img_out = image.Image(image_shape[1], image_shape[0], image.GRAYSCALE,
                          alloc=image.ALLOC_REF, data=result_np)

    # 显示闭操作后的图像 / Show closed image
```

```
    Display.show_image(img_out)

    # 清理内存并输出帧率 / Cleanup and print FPS
    gc.collect()
    print("close fps:", clock.fps())
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **闭运算处理**：调用 `cv_lite.rgb888_close()` 函数执行闭运算，返回处理后的图像 NumPy 数组。
- **图像包装和显示**：将处理结果包装为灰度图像对象，并显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()                          # 停止摄像头 / Stop sensor
Display.deinit()                       # 关闭显示输出 / Deinitialize display
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)  # 设置退出点 / Set safe exit point
time.sleep_ms(100)                     # 稍作延时 / Delay
MediaManager.deinit()                  # 释放媒体资源 / Release media resources
```

参数调整说明

- `kernel_size`：卷积核尺寸，值越大处理的区域范围越大，适合去除较大的噪声；建议使用奇数（如 3、5、7），从 3 开始测试以平衡细节保留和噪声去除。
- `iterations`：迭代次数，值越大闭运算效果越强，但会增加计算量和可能过度平滑图像；建议从 1 开始，根据图像噪声水平逐步增加。
- `threshold_value`：二值化阈值，值越高需要更亮的像素才能被保留，值为 0 时使用 Otsu 自动阈值；建议根据图像亮度从 50 到 150 测试，或设置为 0 以自动适应。